

Frontend Development

A Beginner's Guide

Understanding the Browser · React · Modern Tools

No prior experience required · Diagrams & Code Examples included

- Part 1 — Understanding Frontend Development
 - Part 2 — Introducing React
 - Part 3 — React Fundamentals

Part 1: Understanding Frontend Development ■

What is Frontend Development?

Frontend development is the art of building everything a user **sees and interacts with** in a web browser. Think of a website like a restaurant: the **frontend** is the dining room (tables, menus, ambiance) and the **backend** is the kitchen (cooking the food, managing data). As a frontend developer your job is to make websites look great and feel smooth to use.

The Role of the Browser & The DOM

A web browser (Chrome, Firefox, Safari) reads HTML, CSS, and JavaScript files and draws them on screen. When it loads your HTML it creates a structure called the **DOM (Document Object Model)** — a family tree of every element on the page.

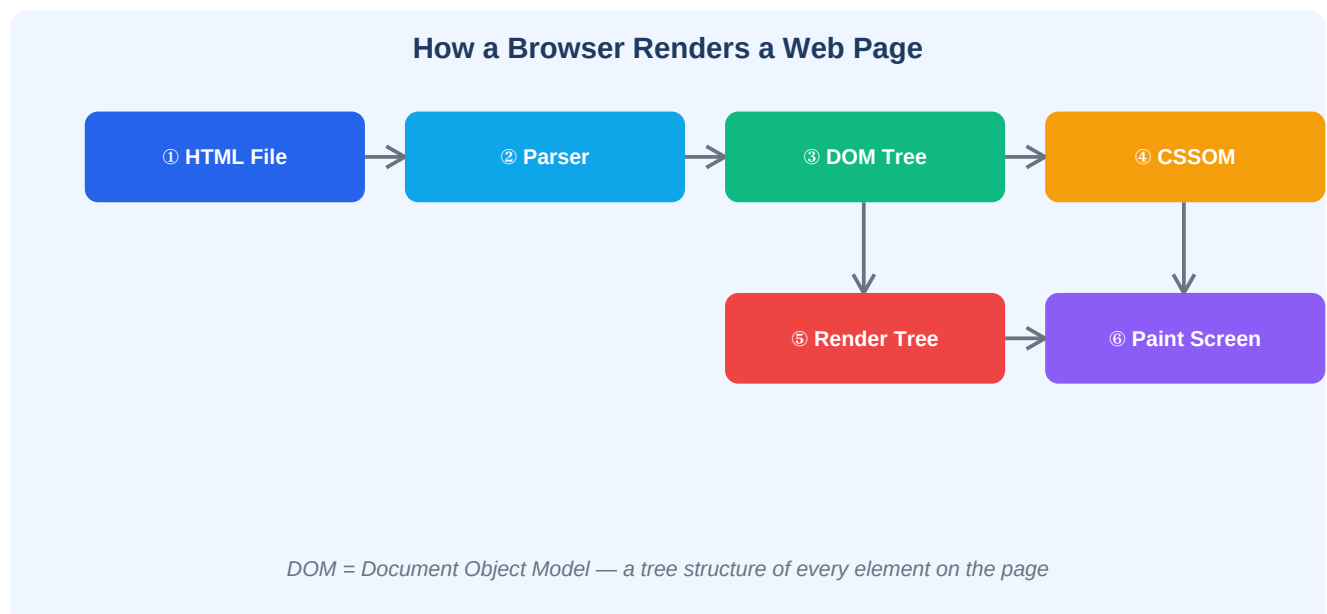


Figure 1: How a browser processes HTML into a visible page

The DOM lets JavaScript *read* and *change* the page dynamically — without reloading. Click a 'Like' button? JavaScript updates the DOM counter instantly.

- **HTML** → Structure (headings, paragraphs, buttons)
- **CSS** → Styling (colors, fonts, layout)
- **JavaScript** → Behavior (clicks, animations, data)

■ Tip

Think of the DOM like a document outline. Every heading, paragraph and image is a node. JavaScript can add, remove or change any node at any time.

Modern Frontend Architectures & Tools

Modern development uses **tools** that speed up building and keep code organised:

Tool	What It Does	Example
Build Tool	Bundles & optimises your code	Vite, Webpack
Package Manager	Installs libraries	npm, yarn
Framework / Library	Provides reusable patterns	React, Vue, Angular
Version Control	Tracks code changes	Git & GitHub
Browser DevTools	Debug & inspect live pages	Chrome DevTools

Single Page Apps (SPA) vs Multi-Page Apps (MPA)

There are two main ways to structure a web application. The difference is *how* the browser loads content when you navigate between pages.

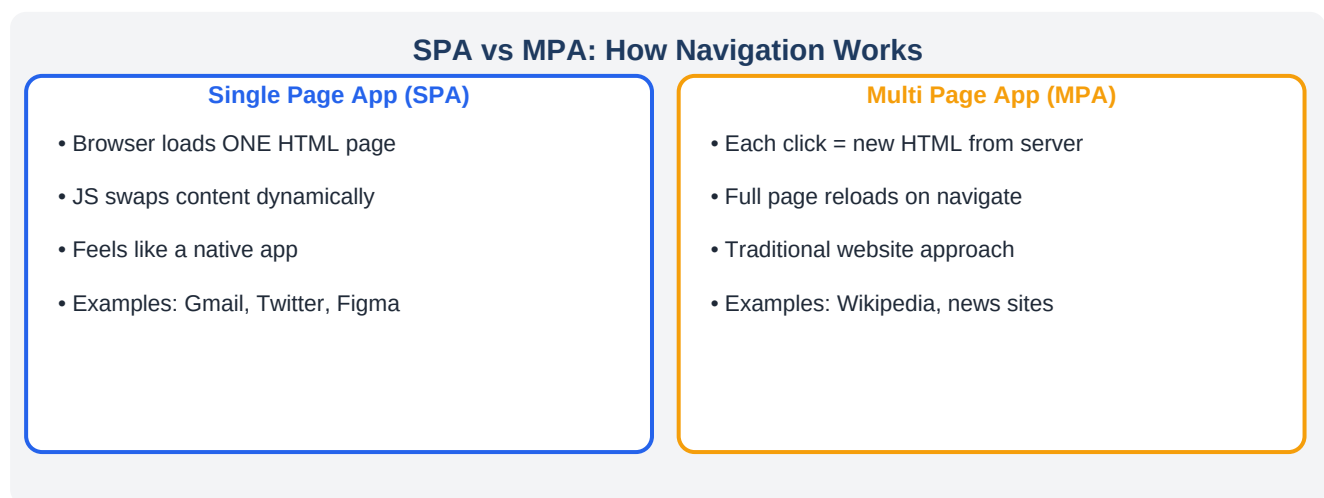


Figure 2: The key difference between SPA and MPA navigation

- **SPA:** Loads once, then updates via JavaScript. Fast and app-like. React is built for SPAs. Examples: Gmail, Twitter, Notion.
- **MPA:** Each page click loads a fresh HTML file from the server. Traditional approach. Examples: Wikipedia, most news sites.

■ Note

When starting out you will most likely build SPAs with React — they feel smoother for users and dominate modern web development.

■ Resources — Frontend Basics

MDN Web Docs — HTML, CSS & JS Reference

<https://developer.mozilla.org>

The most comprehensive web reference. Bookmark it!

The Odin Project — Free Curriculum

<https://www.theodinproject.com>

Free, project-based curriculum — perfect for beginners.

CSS-Tricks

<https://css-tricks.com>

Great articles on CSS layouts and modern frontend patterns.

Part 2: Introducing React ■■

What is React?

React is a **JavaScript library** created by Facebook (Meta) in 2013 for building user interfaces. It is one of the most popular frontend tools in the world. Instead of manually updating the DOM, React lets you build your UI from small, reusable pieces called **components** — like LEGO bricks that snap together.

- **Declarative:** You describe *what* the UI should look like; React figures out *how* to update it.
- **Component-Based:** Build encapsulated components that manage their own logic, then compose them into complex UIs.
- **Learn Once, Write Anywhere:** React also powers mobile apps (React Native) and server-rendered pages (Next.js).

■ Tip

React is NOT a full framework — it focuses on the view layer only. You add other libraries for routing, data fetching, etc.

Setting Up a React Project Using Vite

Vite (pronounced 'veet') is a blazing-fast build tool — the recommended way to start a React project. You'll need **Node.js** installed first (download from nodejs.org).

```
# Step 1: Create a new project
npm create vite@latest my-first-app -- --template react

# Step 2: Enter the project folder
cd my-first-app

# Step 3: Install dependencies
npm install

# Step 4: Start the dev server
npm run dev

# Open http://localhost:5173 in your browser ■
```

Your project folder looks like this:

```

my-first-app/
├── src/
│   ├── App.jsx      ← Your main component
│   ├── main.jsx     ← Entry point (mounts React)
│   ├── index.css    ← Global styles
│   ├── index.html   ← The one HTML file
│   └── package.json ← Project config & dependencies

```

Your First React App

Open `src/App.jsx` and replace everything with this:

```

function App() {
  return (
    <div>
      <h1>Hello, World!</h1>
      <p>My first React app! ■</p>
    </div>
  );
}

export default App;

```

Save the file — the browser updates **instantly** thanks to Vite's Hot Module Replacement (HMR).

What Makes React So Fast? — The Virtual DOM

React uses a technique called the **Virtual DOM**. Instead of updating the slow real browser DOM directly, React first updates a lightweight JavaScript copy, compares it to the previous version, and only updates what actually changed.

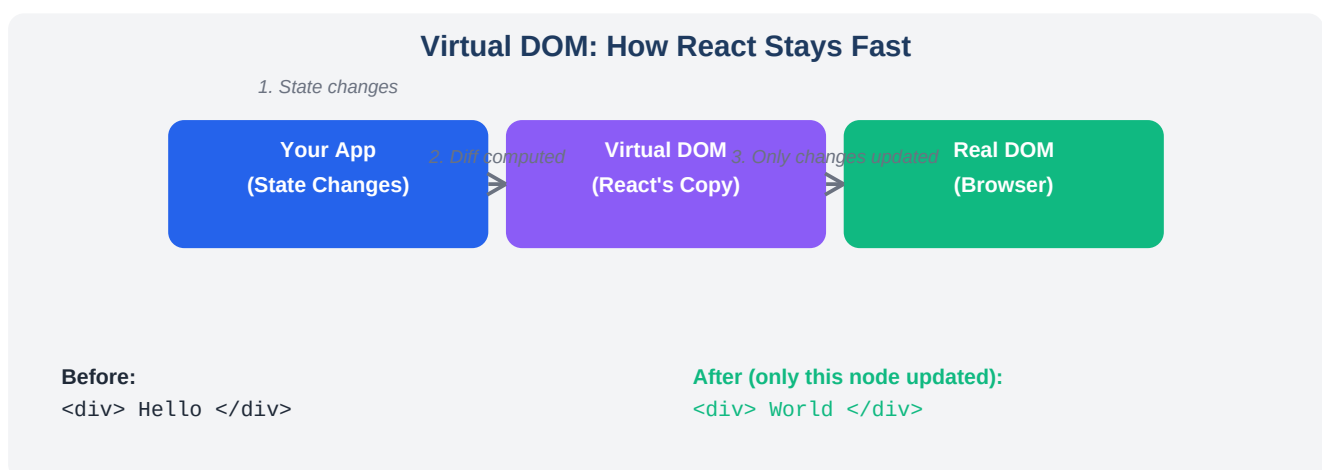


Figure 3: React's Virtual DOM diffing process

- **Batch Updates:** React groups multiple state changes and applies them in one pass.
- **Reconciliation:** React's algorithm to find the minimum number of DOM changes needed.

- **Fiber Architecture:** React's internal engine that keeps animations smooth even under load.

■ Resources — React

Official React Documentation

<https://react.dev>

Best place to learn React — interactive examples and guides.

Vite Documentation

<https://vitejs.dev/guide/>

Learn how to configure and use Vite.

Scrimba — Learn React for Free

<https://scrimba.com/learn/learnreact>

Write code directly in the browser — interactive React course.

Part 3: React Fundamentals ■

Understanding React Components

A **component** is a self-contained piece of UI with its own structure, style, and behaviour. Think of components as custom HTML tags you create yourself. A large app is just many components nested together.

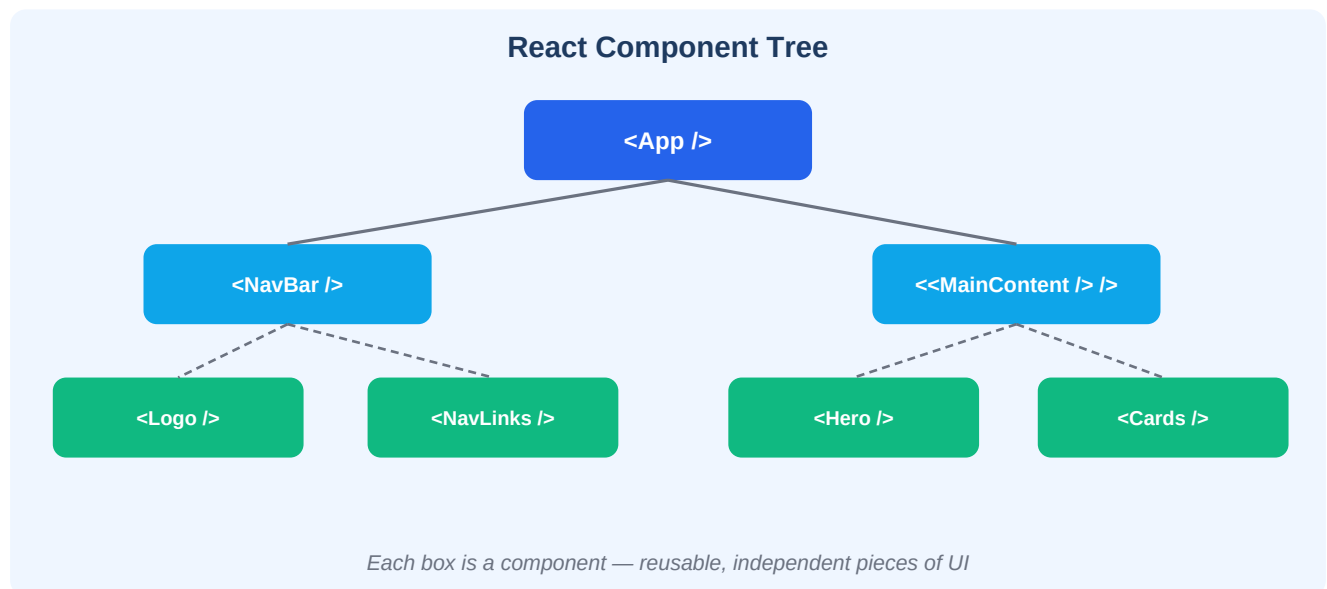


Figure 4: A React app is a tree of components

■ Tip

Rule of thumb: if you find yourself copying and pasting the same UI in multiple places, make it a component!

Class vs. Functional Components

React originally used **Class Components** but now strongly recommends **Functional Components** (introduced with React Hooks in 2019). Functional components are simpler and shorter.

	Functional (Modern ■)	Class (Legacy)
Syntax	A plain JavaScript function	A JavaScript class
State	useState() hook	this.state
Lifecycle	useEffect() hook	componentDidMount() etc.
Code	Shorter & cleaner	More boilerplate
Use today?	YES — always use this	No — avoid for new code

React Elements Using createElement()

Under the hood every JSX expression compiles to **React.createElement()** — a plain JavaScript call that returns an object describing what to render. You will rarely write this directly, but knowing it exists helps you understand what JSX really is.

```
// Under the hood (you rarely write this directly):
const element = React.createElement(
  'h1',                // HTML tag
  { className: 'title' }, // Props
  'Hello, World!'      // Children
);

// Result — a plain JS object:
// { type: 'h1', props: { className: 'title', children: 'Hello!' } }
```

Using JSX

JSX (JavaScript XML) lets you write HTML-like code inside JavaScript. It makes components easy to read. JSX is compiled by Vite to createElement() calls automatically.

```
// JSX — what you write:
function Greeting({ name }) {
  return (
    <div className="greeting">    {/* className not class */}
      <h1>Hello, {name}!</h1>    {/* { } = JavaScript */}
      <p>Welcome to React.</p>
    </div>
  );
}

// JSX Rules:
// 1. Return ONE root element (wrap in <div> or <>...</>)
// 2. Use className instead of class
// 3. Self-close tags: <img /> not <img>
// 4. JavaScript expressions go inside { curly braces }
```

■ Tip

JSX looks like HTML but is JavaScript. Key gotchas: use className (not class) and htmlFor (not for) — those are reserved JS keywords.

Understanding the Lifecycle of a Component

Every React component goes through a **lifecycle**: created (mounted), possibly updated, and eventually removed (unmounted). This helps you know *when* to fetch data, start timers, or clean up resources.

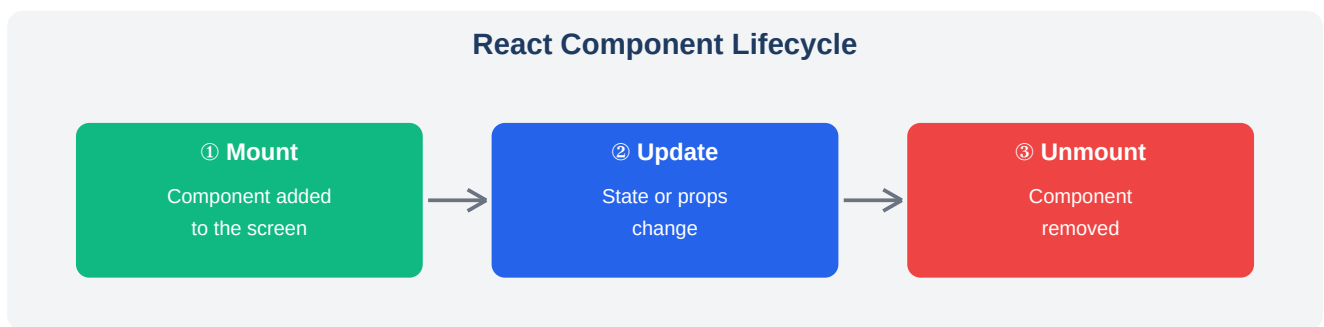


Figure 5: The three phases of a React component's life

```
import { useEffect } from 'react';

function MyComponent() {
  useEffect(() => {
    // MOUNT: runs once when the component appears
    fetchData();
  });

  return () => {
    // UNMOUNT: cleanup when component is removed
    console.log('Cleaned up!');
  };
}, []); // [] = run only on mount

return <div>Hello!</div>;
}
```

Props and Data Flow

Props (properties) are how you pass data from a **parent** to a **child** component. Props are *read-only* — a child must never modify its own props. Data in React always flows **downward** (parent → child).

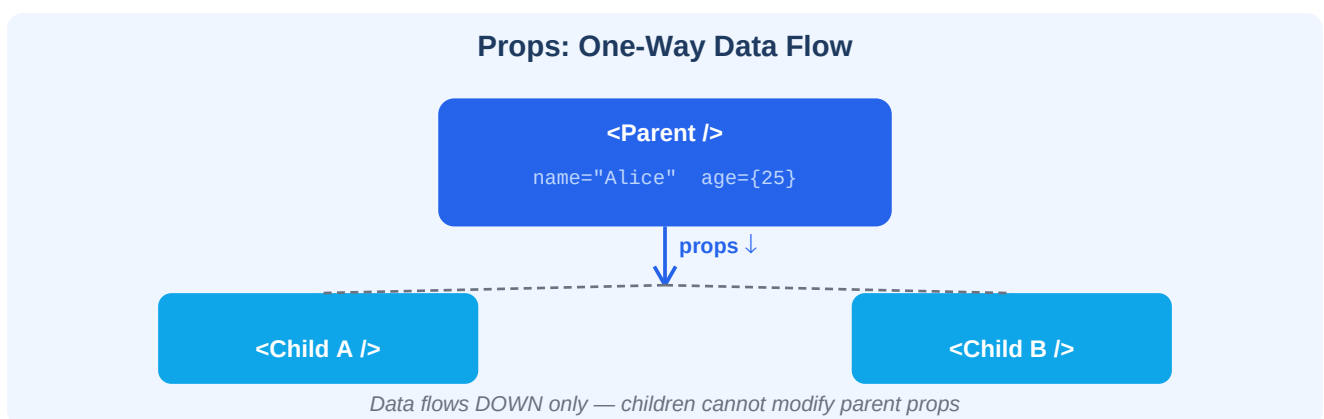


Figure 6: Props flow from parent to child — one direction only

```
// PARENT — passes data as props
function App() {
  return <UserCard name="Alice" age={25} isAdmin={true} />;
}

// CHILD — receives props as a parameter
function UserCard({ name, age, isAdmin }) {
  return (
    <div>
      <h2>{name}</h2>
      <p>Age: {age}</p>
      {isAdmin && <span>Admin ■</span>}
    </div>
  );
}
```

Handling Events & Synthetic Events

React wraps browser events in its own **Synthetic Event** system so they work consistently in all browsers. Event handlers use **camelCase** names and receive a function reference (not a string like in plain HTML).

```
function ClickCounter() {
  const [count, setCount] = React.useState(0);

  function handleClick(event) {
    console.log(event.type); // "click"
    setCount(count + 1);
  }

  return (
    <div>
      <p>Clicked {count} times</p>
      <button onClick={handleClick}>Click me!</button>
    </div>      {/* ↑ camelCase, pass the function — no () */}
  );
}

// Common React events:
// onClick  onChange  onSubmit  onMouseOver  onKeyDown
```

- **onClick** — button clicks
- **onChange** — input / form field changes
- **onSubmit** — form submission
- **event.preventDefault()** — stop default browser behavior (e.g. page reload on form submit)

■■ Project: Building Composable UIs with Functional Components

Put everything together! Build a **User Card List** app that demonstrates components, props, events, and data flow:

```
// 1. UserCard – a single card component
function UserCard({ name, role, onClick }) {
  return (
    <div style={{ border: "1px solid #ccc", padding: "16px", borderRadius: "8px" }}>
      <h3>{name}</h3>
      <p>{role}</p>
      <button onClick={onClick}>View Profile</button>
    </div>
  );
}

// 2. UserList – renders a list of cards
function UserList({ users }) {
  return (
    <div>
      {users.map(user => (
        <UserCard
          key={user.id}           // required for lists!
          name={user.name}
          role={user.role}
          onClick={() => alert(`Viewing ${user.name}`)}
        />
      ))}
    </div>
  );
}

// 3. App – the root component
function App() {
  const users = [
    { id: 1, name: "Alice", role: "Designer" },
    { id: 2, name: "Bob", role: "Developer" },
    { id: 3, name: "Carol", role: "Manager" },
  ];
  return <UserList users={users} />;
}
```

■ Tip

Key concepts practiced: Functional components · Props · `.map()` to render lists · The key prop (required!) · `onClick` event handling.

■ Resources — React Fundamentals

React Tutorial — Tic Tac Toe

<https://react.dev/learn/tutorial-tic-tac-toe>

Build a complete game while learning React. Interactive & beginner-friendly.

React DevTools (Browser Extension)

<https://react.dev/learn/react-developer-tools>

Inspect your component tree live in the browser — essential debugging tool.

JavaScript.info

<https://javascript.info>

Master JavaScript fundamentals (functions, arrays, objects) before going deep into React.

freeCodeCamp — Full React Course (YouTube)

<https://www.youtube.com/watch?v=bMknfKXIFA8>

12-hour comprehensive React course — completely free.

Egghead.io — Beginner's Guide to React

<https://egghead.io/courses/the-beginner-s-guide-to-react>

Short, focused video lessons by React expert Kent C. Dodds.

Happy coding! ■ Every expert was once a beginner. Build things, break things, and keep learning.